

Mechanics of Composite Materials

Project 1

Luncheng Zhang

Kiefer Gonsalves

Word Count: 431 words

Introduction:

For this project we are assigned to complete 3 different questions where we are tasked with calculating and plotting the transformed elastic constants, as well as plotting the shear and biaxial strength of AS4/3501-6-epoxy and E-glass/Epoxy. We calculated each of these materials as unidirectionally across all questions to use the data provided in Table A.6 from the book/campus files. Each of these problems also was a function of the fiber angle, θ , where problems 2.1 and 2.2 are looking at θ from zero to 90 degrees.

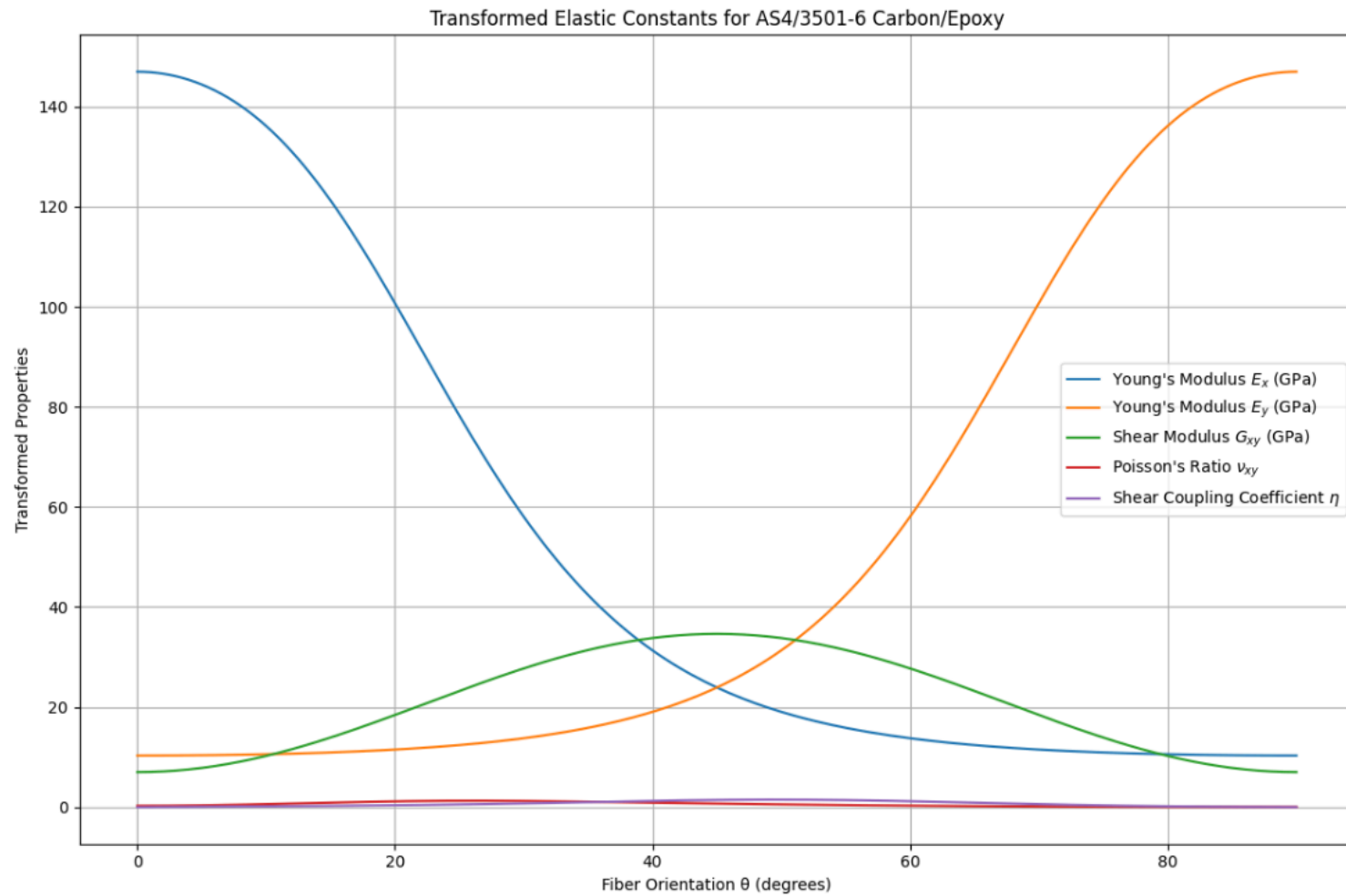
Methods:

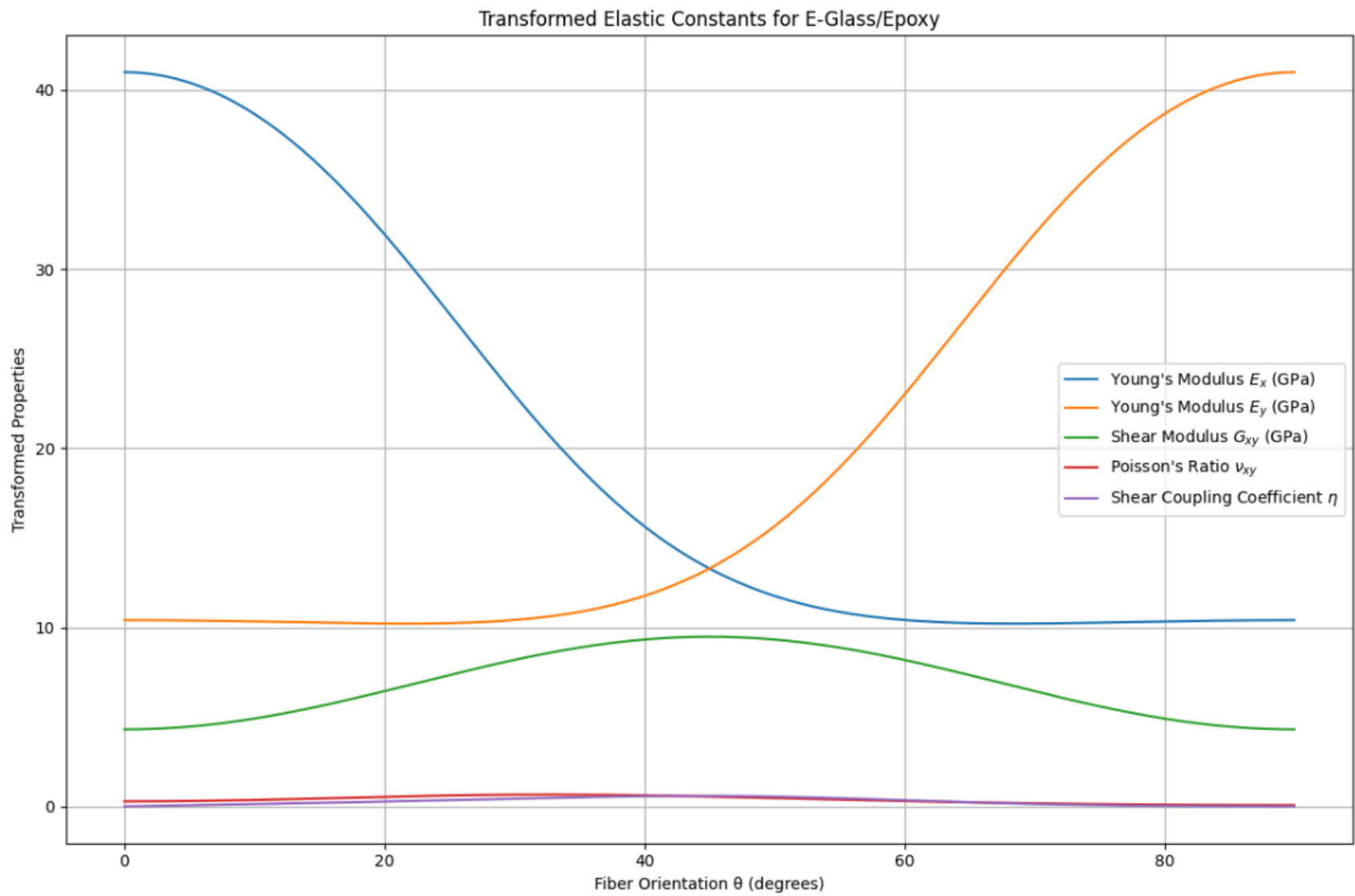
All calculations were implemented using Python with NumPy and Matplotlib. In Part 1, transformed elastic constants—including Young's modulus, shear modulus, major Poisson's ratio, and shear coupling coefficient—were computed as a function of fiber orientation θ by rotating the stiffness matrix Q into $\bar{Q}$.

In Part 2, the Tsai-Hill criterion was applied to calculate shear strength F_s under pure shear load using stress transformation and a bisection method to find the failure point. The Tsai-Wu criterion was used to solve a quadratic equation for biaxial strength F_o under equal biaxial stress. For both parts, results were compared to simplified estimates (F_{2c}) and plotted with percent differences across fiber angles from 0° to 90° .

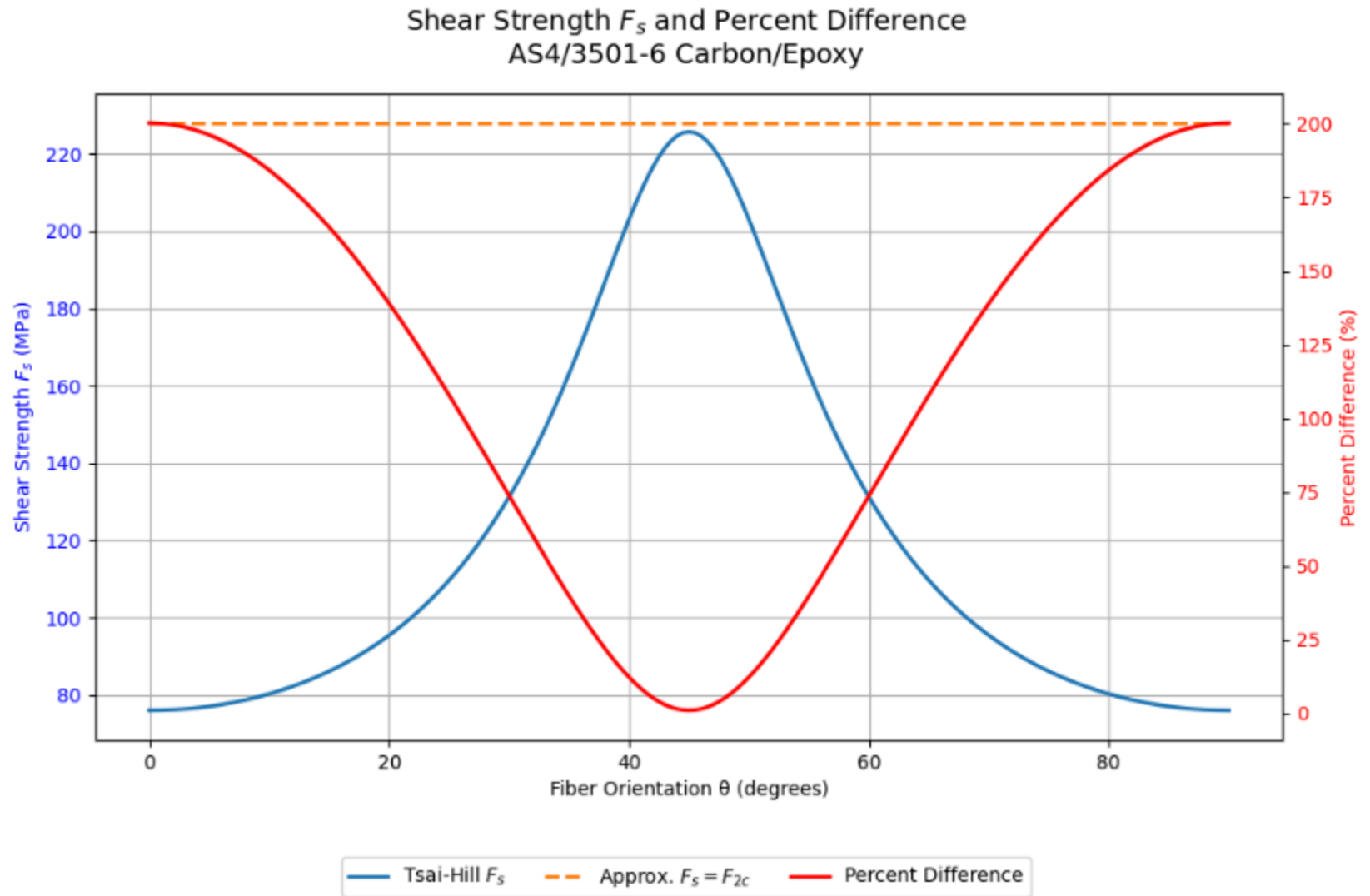
Results:

Q1:

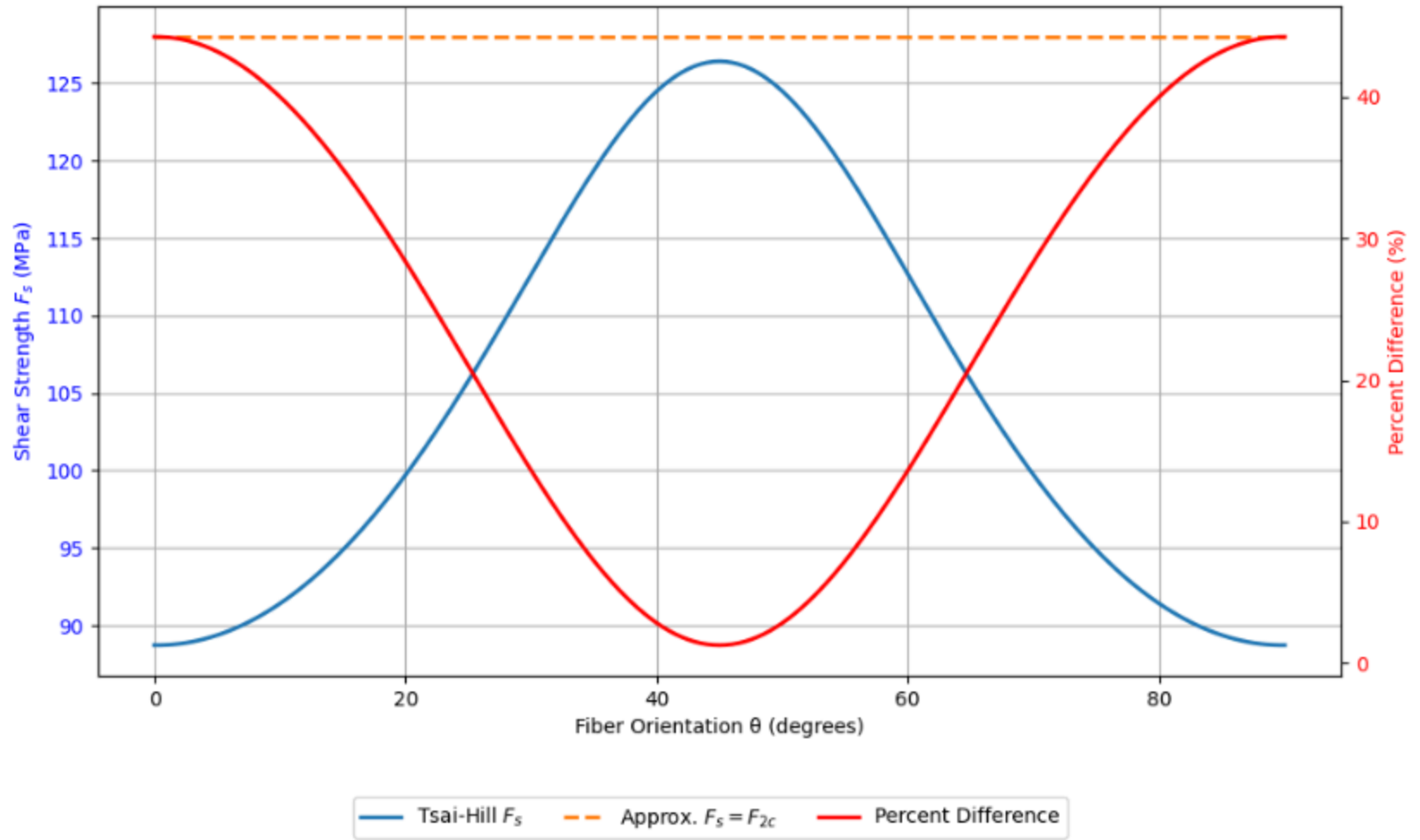




Q2.1:

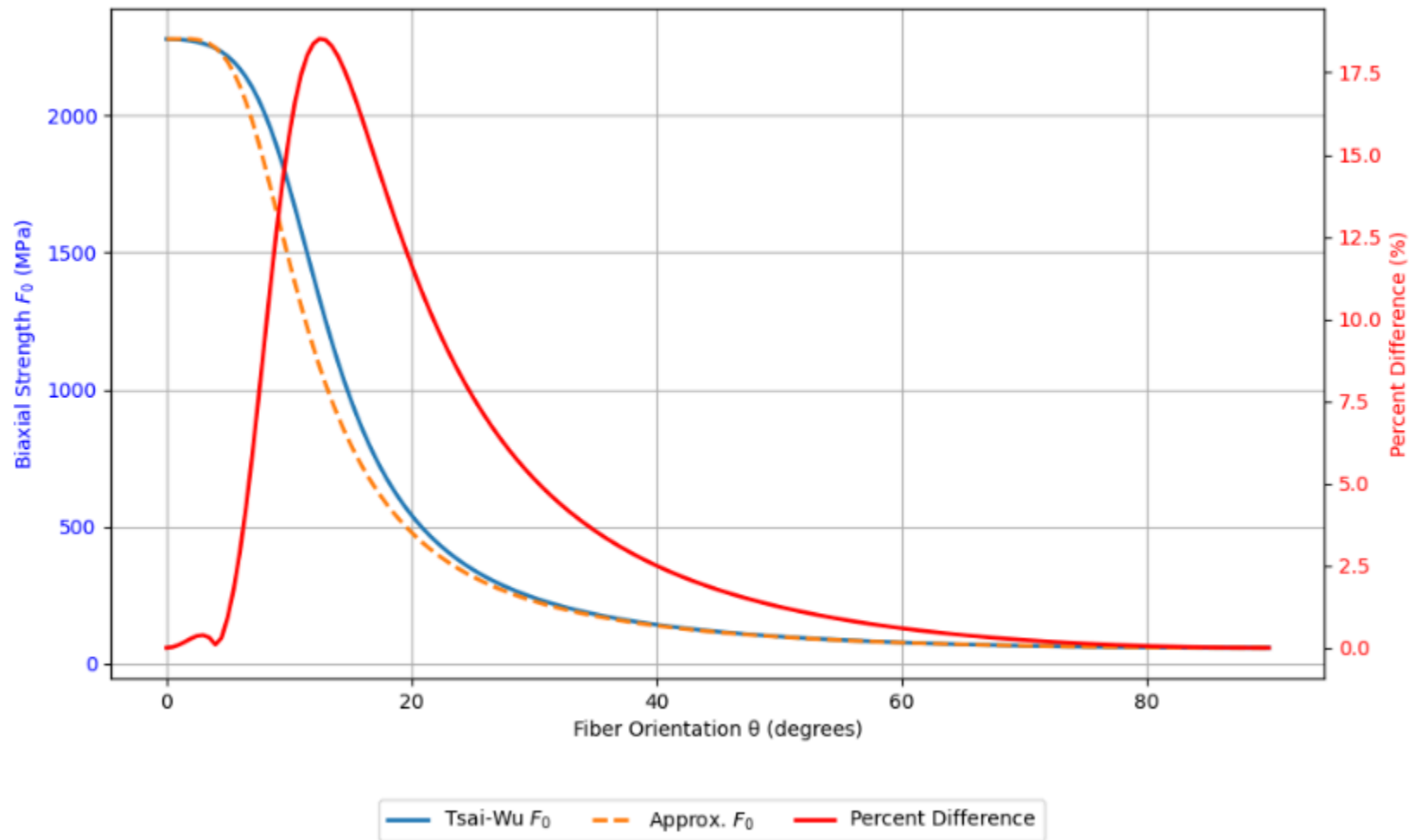


Shear Strength F_s and Percent Difference
E-glass/Epoxy

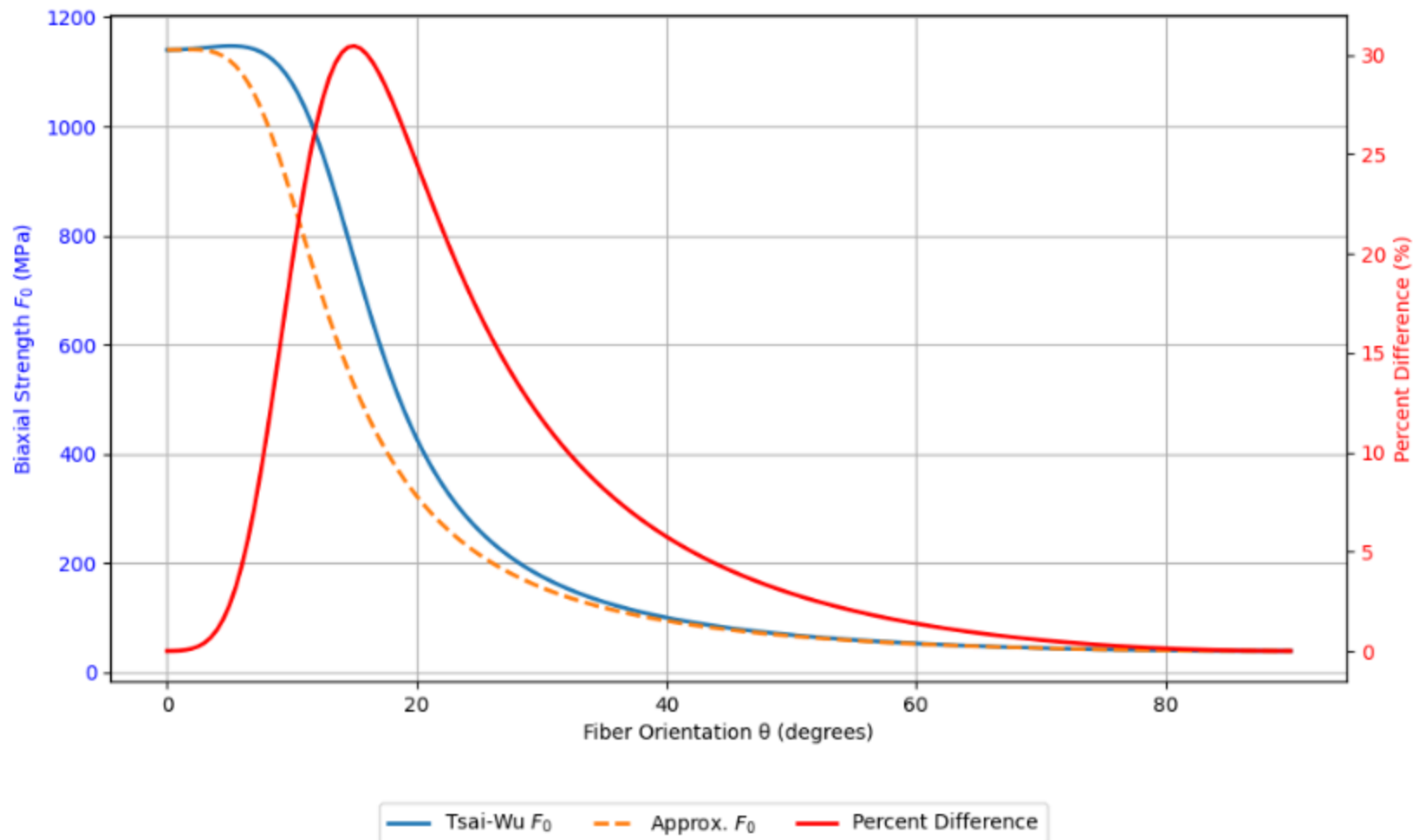


Q2.2:

Biaxial Strength F_0 and Percent Difference
AS4/3501-6 Carbon/Epoxy



Biaxial Strength F_0 and Percent Difference
E-glass/Epoxy



Conclusion:

Overall our project has been successful, as our team was able to properly calculate and plot all of the necessary requirements for AS4/3501-6-epoxy and E-glass/Epoxy. For question one the Young's modulus, shear modulus, major Poisson's ratio, and shear coupling coefficient all seem to be within a reasonable range when comparing both materials as a function of fiber orientation. Question one supports the idea that both materials are anisotropic due to the behavior of how E_x is greatest at zero degrees and how the shear modulus G_{xy} has its local maximum at 45 degrees. Question 2.1 supports that when the materials experience shear stress that the largest F_s is at 45 degrees, this continues to support that both materials are anisotropic. From question 2.1 we can also see how E-glass/Epoxy is weaker than AS4/3501-6-epoxy, but the plots show that as the orientation changes, E-glass/Epoxy is less likely to have its shear strength change. Question 2.2 shows similar data to 2.1, as we can see that both materials are strongest biaxially at zero degrees, but E-glass/Epoxy is more likely to hold its strength as orientation of the fibers change, but is overall weaker than AS4/3501-6-epoxy.

Appendix:

Q1. Code:

```
# 1.
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Material parameters
```

```
carbon_epoxy = {
```

```
    "E1": 147.0, # GPa
```

```
    "E2": 10.3, # GPa
```

```
    "G12": 7.0, # GPa
```

```
    "v12": 0.27,
```

```
}
```

```
e_glass_epoxy = {
```

```
    "E1": 41.0, # GPa
```

```
    "E2": 10.4, # GPa
```

```
    "G12": 4.3, # GPa
```

```
    "v12": 0.28,
```

```
}
```

```
# Q matrix calculation
```

```
def get_Q_matrix(E1, E2, G12, v12):
```

```

v21 = v12 * E2 / E1

denom = 1 - v12 * v21

Q11 = E1 / denom

Q22 = E2 / denom

Q12 = v12 * E2 / denom

Q66 = G12

Q = np.array([

    [Q11, Q12, 0],

    [Q12, Q22, 0],

    [0, 0, Q66]

])

return Q, Q11, Q12, Q22, Q66

```

$\bar{Q}$ transformation and shear-extension coupling coefficient

```
def transform_Q_matrix(Q, Q11, Q12, Q22, Q66, theta_deg):
```

```
    theta = np.radians(theta_deg)
```

```
    m = np.cos(theta)
```

```
    n = np.sin(theta)
```

```
    m2, n2 = m**2, n**2
```

```
    m4, n4 = m2**2, n2**2
```

```
    Qbar11 = Q11 * m4 + 2*(Q12 + 2*Q66)*m2*n2 + Q22 * n4
```

$$Q_{bar22} = Q_{11} * n^4 + 2*(Q_{12} + 2*Q_{66})*m^2*n^2 + Q_{22} * m^4$$

$$Q_{bar12} = (Q_{11} + Q_{22} - 4*Q_{66})*m^2*n^2 + Q_{12}*(m^4 + n^4)$$

$$Q_{bar66} = (Q_{11} + Q_{22} - 2*Q_{12} - 2*Q_{66})*m^2*n^2 + Q_{66}*(m^2 - n^2)**2$$

$$Q_{bar16} = (Q_{11} - Q_{12} - 2*Q_{66})*m**3*n - (Q_{22} - Q_{12} - 2*Q_{66})*m*n**3$$

```
Qbar_simple = np.array([
    [Qbar11, Qbar12, 0],
    [Qbar12, Qbar22, 0],
    [0,    0,    Qbar66]
])
```

```
return Qbar_simple, Qbar16
```

```
# Compute transformed properties including Ex, Ey, Gxy, vxy, eta
```

```
def get_transformed_properties(material_dict, theta_deg):
```

```
    Q, Q11, Q12, Q22, Q66 = get_Q_matrix(
        material_dict["E1"],
        material_dict["E2"],
        material_dict["G12"],
        material_dict["v12"]
    )
```

```
    Qbar_simple, Qbar16 = transform_Q_matrix(Q, Q11, Q12, Q22, Q66, theta_deg)
```

```
    Sbar = np.linalg.inv(Qbar_simple)
```

```

Ex = 1 / Sbar[0, 0]
Ey = 1 / Sbar[1, 1]
Gxy = 1 / Sbar[2, 2]
vxy = -Sbar[0, 1] / Sbar[0, 0]
eta = Qbar16 / Ex

```

```

return Ex, Ey, Gxy, vxy, eta

```

```

# Plotting all 5 properties

```

```

def plot_transformed_constants(material_dict, material_name):

```

```

    angles = np.linspace(0, 90, 181)

```

```

    Ex_list, Ey_list, Gxy_list, vxy_list, eta_list = [], [], [], [], []

```

```

    for theta in angles:

```

```

        Ex, Ey, Gxy, vxy, eta = get_transformed_properties(material_dict, theta)

```

```

        Ex_list.append(Ex)

```

```

        Ey_list.append(Ey)

```

```

        Gxy_list.append(Gxy)

```

```

        vxy_list.append(vxy)

```

```

        eta_list.append(eta)

```

```

plt.figure(figsize=(12, 8))

```

```

plt.plot(angles, Ex_list, label="Young's Modulus  $E_x$  (GPa)")
plt.plot(angles, Ey_list, label="Young's Modulus  $E_y$  (GPa)")
plt.plot(angles, Gxy_list, label="Shear Modulus  $G_{xy}$  (GPa)")
plt.plot(angles, vxy_list, label="Poisson's Ratio  $\nu_{xy}$ ")
plt.plot(angles, eta_list, label="Shear Coupling Coefficient  $\eta$ ")
plt.xlabel("Fiber Orientation  $\theta$  (degrees)")
plt.ylabel("Transformed Properties")
plt.title(f"Transformed Elastic Constants for {material_name}")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```

```

# Run for both materials

```

```

plot_transformed_constants(carbon_epoxy, "AS4/3501-6 Carbon/Epoxy")
plot_transformed_constants(e_glass_epoxy, "E-Glass/Epoxy")

```

Q2.1 Code:

```
# 2.1

import numpy as np

import matplotlib.pyplot as plt

# Material strength data

materials = {

    "AS4/3501-6 Carbon/Epoxy": {

        "F1t": 2280, "F1c": 1725, "F2t": 57, "F2c": 228, "F6": 76

    },

    "E-glass/Epoxy": {

        "F1t": 1140, "F1c": 620, "F2t": 39, "F2c": 128, "F6": 89

    }

}

# Tsai-Hill shear strength function

def calculate_Fs_tsai_hill(F1t, F1c, F2t, F2c, F6, theta_deg):

    theta = np.radians(theta_deg)

    m = np.cos(theta)

    n = np.sin(theta)

    sigma1 = 1.0

    sigma2 = -2 * m * n * sigma1
```

```
tau12 = (m**2 - n**2) * sigma1
```

```
F1 = F1t if sigma1 >= 0 else F1c
```

```
F2 = F2t if sigma2 >= 0 else F2c
```

```
a = (sigma1 / F1)**2 - (sigma1 * sigma2) / (F1**2)
```

```
b = (sigma2 / F2)**2
```

```
c = (tau12 / F6)**2
```

```
tau0 = 1 / np.sqrt(a + b + c)
```

```
return tau0
```

```
# Approximate shear strength function
```

```
def calculate_Fs_approx(F2t, F2c, theta_deg):
```

```
    return F2c
```

```
# Evaluation range
```

```
angles = np.linspace(0, 90, 181)
```

```
# Plotting loop
```

```
for material_name, props in materials.items():
```

```
    Fs_full = []
```

```
    Fs_approx = []
```

```
percent_diff = []
```

```
for theta in angles:
```

```
    Fs = calculate_Fs_tsai_hill(
```

```
        props["F1t"], props["F1c"],
```

```
        props["F2t"], props["F2c"],
```

```
        props["F6"], theta
```

```
    )
```

```
    Fs_a = calculate_Fs_approx(props["F2t"], props["F2c"], theta)
```

```
    diff = 100 * abs(Fs - Fs_a) / Fs
```

```
    Fs_full.append(Fs)
```

```
    Fs_approx.append(Fs_a)
```

```
    percent_diff.append(diff)
```

```
fig, ax1 = plt.subplots(figsize=(10, 6))
```

```
# Left Y-axis: Shear strength
```

```
line1, = ax1.plot(angles, Fs_full, label="Tsai-Hill  $F_s$ ", linewidth=2)
```

```
line2, = ax1.plot(angles, Fs_approx, '--', label="Approx.  $F_s = F_{2c}$ ", linewidth=2)
```

```
ax1.set_xlabel("Fiber Orientation  $\theta$  (degrees)")
```

```
ax1.set_ylabel("Shear Strength  $F_s$  (MPa)", color='b')
```

```
ax1.tick_params(axis='y', labelcolor='b')
```

```

ax1.grid(True)

# Right Y-axis: Percent difference

ax2 = ax1.twinx()

line3, = ax2.plot(angles, percent_diff, 'r', label="Percent Difference", linewidth=2)

ax2.set_ylabel("Percent Difference (%)", color='r')

ax2.tick_params(axis='y', labelcolor='r')

# Combine all legends and place below the plot

lines = [line1, line2, line3]

labels = [line.get_label() for line in lines]

fig.legend(lines, labels, loc='lower center', ncol=3, bbox_to_anchor=(0.5, -0.1))

fig.suptitle(f"Shear Strength  $F_s$  and Percent Difference\n{material_name}", fontsize=14)

plt.tight_layout()

plt.subplots_adjust(bottom=0.1) # Add extra space for the legend at the bottom

plt.show()

```

Q2.2 Code:

```
# 2.2

import numpy as np

import matplotlib.pyplot as plt


# Material strength data

materials = {

    "AS4/3501-6 Carbon/Epoxy": {

        "F1t": 2280, "F1c": 1725, "F2t": 57, "F2c": 228

    },

    "E-glass/Epoxy": {

        "F1t": 1140, "F1c": 620, "F2t": 39, "F2c": 128

    }

}


# Calculate the Tsai-Wu coefficients

def tsai_wu_coefficients(F1t, F1c, F2t, F2c):

    f1 = (1 / F1t) - (1 / F1c)

    f2 = (1 / F2t) - (1 / F2c)

    f11 = 1 / (F1t * F1c)

    f22 = 1 / (F2t * F2c)

    f12 = -0.5 * np.sqrt(f11 * f22) # empirical assumption

    return f1, f2, f11, f22, f12
```

```

# Full Tsai-Wu biaxial strength F0

def calculate_F0_tsai_wu(F1t, F1c, F2t, F2c, theta_deg):

    theta = np.radians(theta_deg)

    m = np.cos(theta)

    n = np.sin(theta)

    f1, f2, f11, f22, f12 = tsai_wu_coefficients(F1t, F1c, F2t, F2c)

    f1_theta = f1 * m**2 + f2 * n**2

    f11_theta = f11 * m**4

    f22_theta = f22 * n**4

    f12_theta = f12 * m**2 * n**2

    a = f11_theta + f22_theta + 2 * f12_theta

    b = f1_theta

    Fo = (-b + np.sqrt(b**2 + 4 * a)) / (2 * a)

    return Fo

# Approximate F0 for high-strength composite

def calculate_F0_approx(F1t, F1c, F2t, F2c, theta_deg):

    theta = np.radians(theta_deg)

```

```

m = np.cos(theta)

n = np.sin(theta)


F1 = F1t # assumption for tensile failure
F2 = F2t # assumption for tensile failure


return 1 / np.sqrt((m**2 / F1)**2 + (n**2 / F2)**2)


# Evaluation range
angles = np.linspace(0, 90, 181)


# Plotting loop
for material_name, props in materials.items():

    F0_full = []

    F0_approx = []

    percent_diff = []


    for theta in angles:

        Fo = calculate_F0_tsai_wu(

            props["F1t"], props["F1c"],

            props["F2t"], props["F2c"],

            theta

        )

```

```

Fo_approx = calculate_F0_approx(
    props["F1t"], props["F1c"],
    props["F2t"], props["F2c"],
    theta
)

diff = 100 * abs(Fo - Fo_approx) / Fo

F0_full.append(Fo)
F0_approx.append(Fo_approx)
percent_diff.append(diff)

fig, ax1 = plt.subplots(figsize=(10, 6))

# Left Y-axis: Biaxial strength
line1, = ax1.plot(angles, F0_full, label="Tsai-Wu $F_0$", linewidth=2)
line2, = ax1.plot(angles, F0_approx, '--', label="Approx. $F_0$", linewidth=2)
ax1.set_xlabel("Fiber Orientation  $\theta$  (degrees)")
ax1.set_ylabel("Biaxial Strength $F_0$ (MPa)", color='b')
ax1.tick_params(axis='y', labelcolor='b')
ax1.grid(True)

# Right Y-axis: Percent difference
ax2 = ax1.twinx()

```

```

line3, = ax2.plot(angles, percent_diff, 'r', label="Percent Difference", linewidth=2)

ax2.set_ylabel("Percent Difference (%)", color='r')

ax2.tick_params(axis='y', labelcolor='r')


# Combine legends

lines = [line1, line2, line3]

labels = [line.get_label() for line in lines]

fig.legend(lines, labels, loc='lower center', ncol=3, bbox_to_anchor=(0.5, -0.1))


fig.suptitle(f"Biaxial Strength  $F_0$  and Percent Difference\n{material_name}", fontsize=14)

plt.tight_layout()

plt.subplots_adjust(bottom=0.1)

plt.show()

```